

If you asked me about AI a year ago, I would have told it is all hype. Can't do anything real. Yeah, that was naive. But, in my defense, I did play with LLMs and the results were... uninspiring. Tried writing code and it always failed. Tested how well it could write stories and it flopped. So bad, but things have changed. First, LLMs and tooling around them got way better. Second, I learned how to effectively use LLMs and using LLMs effectively is a skill we all need in 2026.

Let's get one thing out of the way. I will never put my name on an article, story, course, or anything else written by AI undisclosed. This article is 100% hand-written by me, but the image is from NanoBanana.

Not sure about you, but I love learning how things work. How to improve. How to better use tools, and AI is just another tool. Starting out, it sucked. Claude. GPT. Gemini. Random no-code sites. All of it sucked for me, but guess what. As usual, the tool wasn't the problem. I was. Or more specifically, my skill with the tool was the problem.

Now, we should be clear. This isn't The Ultimate Guide to Claude Code and I will not 10X Your LLM Productivity in Three Easy Steps. If I ever write titles like that, you have my explicit permission to slap me. Hard. I'd deserve it. Like any other skill, it takes time, effort and experimentation get good with LLMs. Today, we will learn how to get started and what to focus on. Think of this as the context you need to begin your journey.

The Story — If You've Read Me, You Know It Is Coming

So there I was. Just accepted an offer at Shopify and read about the CEO and how the company uses AI. A bit intimidating if I'm being honest. So, I dug into it a bit more. Got a Claude Pro subscription and started building a website — a complicated one. (Yes, I use em dashes and will not stop just because AI overuses them just like me.) Day one, all joy. Wow, this is actually working. Day two?

Man, this is horrible. You said this is production ready. You're absolutely right. I did not write tests that verify it works. Not a great start. But I knew my new job encouraged LLM use, so... Kept going. Restarted a bunch of times over a month or so and got to a point where things were working. We could add features, refactor, change things way faster than I ever could. Yup, I'm a data engineer, not a web developer. Good progress. But what's next.

I've wanted to create an SQL course for people on the business side for a long time. Intuitive SQL — From 0 to Dangerous. Let's do that next. Claude should be able to help me. Fun first day. Then, I thought, hmm. My data generator is pretty basic. I should create a better one. Something that can create any data, with patterns, DQ issues. That's a fun Sunday afternoon project. See where this is going?

Eight months and an upgrade to Claude Max5 later, I have a really nice synthetic data generator. Throughout that time, I experienced the famous summer Claude bugs, many complete re-architectures of my project (good synthetic data is hard!), and many successes at

home and at work. Needless to say, I learned a ton, but the most important is that context is king.

Context Is Everything — The Goldilocks Zone

It wasn't long ago that we kept hearing about prompt engineering, and it made sense with my layman's understanding of transformers. Start with words that will be closer to the ones you want. "You are an expert data engineer, specializing in..." or my favorite, "You are a psychotic data analyst who tracks down issues with the precision of a serial killer." This one got Claude on Cursor to do more work and really track down issues. These tricks used to be needed, but now they are mostly a novelty.

LLMs are simply better now and the prompt is far less important than the context you give it. But what does that mean? Why do we care? We care because the context you provide to an LLM is the difference between great results and constant frustration. Every LLM has a limit. How many tokens they can keep in a session. For most Claude models it is 200K tokens. Some days that seems like a lot. Some days it seems like so little. But here's the secret. Keeping context to less than about 2/3 of that provides far better results.

Imagine if I spend thirty minutes explaining to you the glorious cheese that is the band Krokus. Then another thirty minutes about how great Noble Raman in Keene, NH is. Trust me, that chef is amazing! We chat about our favorite methods of making coffee, then I ask you, "Which classic Sweet song did Krokus amazingly cover?" Ignoring that you never wanted to hear about Krokus or which songs they covered in the first place, would you remember? Probably not.

Cover of Ballroom Blitz by Sweet by the Awesome Cheesiness that Is Krokus

LLMs are similar in that they have a limited capacity to remember and differentiate between what is important and what is not. Let's look at a problem I ran into. This was an extremely frustrating weekend of poor results from my process.

I had spent hours and hours over several days creating architecture documents for what I wanted to build. Then I told Claude to build a small portion of it in a sprint. Claude read the docs, built the small portion... and a bunch of broken, unfinished work we weren't ready to build yet. But wait, my prompt told it exactly what to build, why did it try to build more? And why didn't it follow the explicit instructions I gave it?

I asked Claude what went wrong and it said, of course, You're absolutely right, I.... Seeing a theme here, yet? I had explicit instructions and did not follow them... I had given Claude — a giant people-pleasing machine — a bunch of context about the project, then asked to deliver only a portion of it. Do you know any people pleasers who wouldn't try to do more?

When you structure your context documents with a high-level index and more detailed docs it can read when needed, you will get better results. When you create processes that limit context

to exactly what is needed, you will see amazing results. But how do you do that? That's the real question. Let me give you a snippet in my implement-sprint skill.

Rule 1: Orchestrator Reads Minimally

You (the orchestrator) read ONLY these files:

```
| File | Purpose |  
|-----|-----|  
| `docs/sprints/current/spec.md` | Sprint spec (required) |  
| `docs/sprints/current/state.yaml` | Phase status (required) |
```

****Do NOT read source files, architecture docs, config models, or any `.py` file.**** The implementer agent reads what it needs. Your job is orchestration, not comprehension. In this skill, the main session acts as the orchestrator. It only needs to understand what to implement in the sprint. It does not need to read architecture docs or understand the features of the product. So this skill helps it read exactly what it needs and no more.

Not only is this more efficient, but it keeps Claude on track and actually completes the entire process autonomously. Before adding this rule, Claude would spend five minutes reading just about everything in my project. By the time it got around to orchestrating the sprint, it had already lost the plot.

On the other hand, if you don't provide Claude with good instructions on how you want work done, it will have to make it up. Do you have specific testing practices? Architectural preferences? That needs to be part of your context. Put them in CLAUDE.md or in a separate doc with a link in CLAUDE.md. Link it in your specific skill. Let Claude know.

It's like Goldilocks and the Three Bears. You don't want the porridge too hot or too cold. You want it just right. Spend time with your documentation. Continually prune it. Heck, ask Claude to do it. Haiku is lightning fast when it comes to editing docs and Opus is very thorough.

If you take only one thing away from this article it should be this. Context management is the difference between success and failure.

When It Isn't Working — May the Odds Be Ever in Your Favor

What about when things just aren't working. We've all read the myriad posts from people who choose not to learn how Claude works. "Anyone notice Claude is dumb today?" "Claude is stupid as sh**t suddenly"... .. It's really frustrating to read, because most of these people could fix their problems in less time than it takes to write and respond to Reddit posts. They choose not to.

Remember, LLMs are stochastic machines full of randomness. They are not scripts that run a defined set of instructions. Understand this. Internalize it. The same prompt will often achieve different results, and there are three primary levers Anthropic has that will change results.

Model changes: New models almost always behave differently. That work you did easily on 4.5, now it struggles in 4.6. But 4.6 does other things way better. Better model, but it wrecked your process.

Tooling: Changes to Claude Code or the desktop app change how we interact with the model. Something like auto plan mode when your /commands expect a clean context can wreck your process.

Backend. We don't have much visibility here, but any changes in the backend can change results. The summer dumb down is an example of bugs making things worse. Other changes might make things better but wreck your current process.

That's just Anthropic. You have tons of ways to impact the randomness.

CLAUDE.md updates

New docs

New code

Adding MCPs

Change to your prompt or skill

Copying that giant PDF into your project

etc.

All of these and more might require you to change how you work with the tool. When things worked yesterday but not today it is far more likely that something intentionally changed than Claude suddenly became dumb. Your process must change, but here's the good news.

Claude is really good at figuring out what went wrong and how to fix it. Didn't write tests? Ask it why. Didn't follow instructions? Ask it why. Stopped working before all tasks were done? You guessed it. Ask it why. And here's a little trick.

Claude will often focus on what it did wrong instead of how to fix it. Remember when I said prompt engineering is largely unnecessary? That was a lie. There are times when a certain prompt yields better results. If you find Claude focusing on what went wrong instead of how to fix it, say, "I believe in blaming processes not people or LLMs. Let's figure out why this happened." Bam!

Full fixing mode. I used this when my implement-sprint skill kept running out of context before finishing. Opus 4.6 WANTS to do more work and uses a ton of context doing it. Knowing that all session logs are stored in ~/.claude/I just asked Claude to analyze a few sessions. In a few hours, I cut down token usage by about 50% and created a useful skill to make it faster in the future. Now I analyze sessions for all kinds of reasons and quickly optimize my processes.

What You Can Do — Practical Tips

Let's keep this simple. Just a list of tips that helped me out.

Managing Context

Make it easy for Claude to find the needed information. List specific files and directories in CLAUDE.md and/or in .claude/skills.

Remove duplication in your docs. It is too hard to keep them in sync and will provide conflicting information.

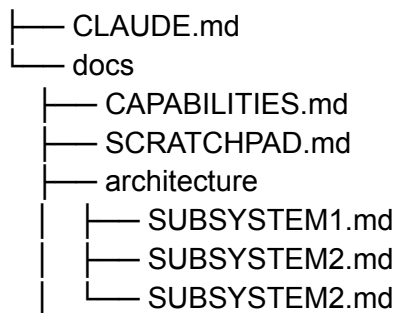
Code is truth. No need to explain how your code works in the docs. Just link to the actual code. Keep docs high level. Purpose. Reasoning. Algorithms. Flows.

Keep CLAUDE.md as small as is reasonable. Only the stuff that every session needs. If some sessions will not need it, put it somewhere else. Stick to repo overview, file structure, important principles and invariants.

Continuously prune your docs. Review them. Edit them. Remove anything that isn't needed.

Tell Claude "These docs must be optimized for LLM use and token efficiency"

Here's an example layout



Getting Shit Done

Never use an LLM for automatable tasks. Use the LLM to create a script for the task.

Create reusable skills. If you have to type it in over and over, put it in a skill.

Iterate. After Claude does work, review it and provide feedback. Or ask Claude to review it.

Work, review, improve. Sometimes it takes several instances of that cycle to get things right.

git reset — hard is your friend. Sometimes a session just doesn't work. That's OK. Remember, stochastic machine? Just reset and try again.

Custom subagents have their own separate context window. Long processes should make good use of them.

/clear is your friend. /compact is not. By the time you compact, your session is already in bad shape. It's time to think about smaller units of work or skills that use custom subagents.

When things stop working, ask Claude to analyze your session, then fix your process.

Jumping Into a New Repo

Ask Claude to analyze the repo and create human readable docs. In a large, complex repo, this could be several hours or even a few day's work. I learned so much about the primary repo at a new job doing this.

Then ask it to create LLM docs. Specifically say "These docs must be optimized for LLM use and token efficiency"

Ask Claude lots of questions to learn more.
My Son Wants to Get Into “IT” or “Cyber Security”
Here’s How He’ll Do Things Different
leo-godin.medium.com

What About Frameworks? The Magic 10X?

There are more and more new frameworks for working with LLMs every day. Open Spec and BMAD are very popular. I built a website with a modified version of Spec Kitty. It was really nice, especially early on in the project. These are great tools and I encourage you to try them out. See what works for you.

In the end, I don’t use them. It’s just a personal preference, but I feel the docs end up too spread out. We have nicely defined features, designs and architecture decisions, but it is difficult to understand the current state of all three. For new projects, I use my own stuff built up over the past year. This provides two main benefits.

I get exactly what I want for how I want to work. My process is lighter weight. Instead of planning everything to the specific task level, this stays one level up. I like to use repetition to plan/review, implement/review until things are in great shape. Again, it’s just a personal preference. More importantly for you, evolving this over time helped me learn more about how Claude behaves. It forced me to experiment and dig deeper. It taught me how to effectively use LLMs. Just grabbing a framework is great to get work done, but decreases your learning. Then, when the model changes, or something isn’t working, you are stuck. Starting over from scratch. In the end, it’s really a matter of priorities and goals just like learning any skill. If you want to learn Python, don’t have an LLM write all your code. Learn Python, then use the LLM to help you learn and improve. If you want to learn how to effectively use LLMs, you’ll need to do a bit of work.

Damn, You’re Still Writing? C’mon Man. I Only Have so Much Time

Using LLMs effectively is a skill we all need in 2026, and just like any other skill, it takes time and effort to improve. By managing context and making it easier for the LLM to get just the right amount of information, we greatly improve results. No ultimate guide or 10x workflow will teach you everything. So, dig in. Learn. Do. Then learn some more. Before you know it, you’ll be flying through PRs, bug fixes and troubleshooting production issues faster than a raptor on crack.